



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-32280 Go crypto/x509 chain-building denial-of-service risk

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-09
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-32280 Go crypto/x509 chain-building denial-of-service risk - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-09 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-32280 Go crypto/x509 chain-building denial-of-service risk

1. Executive Summary

CVE-2026-32280 is a high-severity denial-of-service flaw in Go's standard-library `crypto/x509` certificate-verification logic. During certificate-chain building, the amount of work performed is not correctly bounded when a verifier is given a large set of intermediate certificates through `VerifyOptions.Intermediates`. The result is excessive verification work that can be triggered over the network, impacting both direct users of `crypto/x509` and applications that rely on `crypto/tls`.

The practical consequence is availability loss rather than code execution or data theft. Any Go application that validates attacker-influenced certificate chains can spend disproportionate CPU time processing candidate intermediates. Exposure is therefore highest for Go TLS clients, security tooling, proxies, agents, service-mesh components, or custom applications that repeatedly validate untrusted certificates and have not been rebuilt on the fixed releases.

2. Vulnerability Metadata

Field	Value
CVE	`CVE-2026-32280`
Go advisory ID	`GO-2026-4947`
Product / component	Go standard library `crypto/x509` with impact extending to `crypto/tls`
Affected routines	`Certificate.buildChains` and `Certificate.Verify`
Weakness classification	`CWE-770: Allocation of Resources Without Limits or Throttling`
CVSS v3.1	`7.5 / HIGH`
CVSS vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
Affected version branches	Versions before `Go 1.25.9`, and `Go 1.26.0` through before `Go 1.26.2`
Fixed versions	`Go 1.25.9` and `Go 1.26.2`
Public disclosure date	`2026-04-08`
Upstream tracking	Go issue `#78282`
Credit	Jakub Ciolek

3. Source Review & Confidence

Source	Contribution	Confidence
MSRC Update Guide	Intake trigger and high-level Microsoft tracking	High
Go CNA CVE 5.2 record	Canonical description, affected semver ranges, impacted routines, references, and `CWE-770`	High
Go advisory `GO-2026-4947`	Affected package and fixed-version confirmation	High

NVD JSON 2.0	Severity, vector, and publication metadata	High
Go issue `#78282`	Upstream exploit-mechanics framing for excessive chain-building work	High
Go release announcement / history	Confirms fixes in `1.25.9` and `1.26.2`	High
Debian tracker	Downstream package exposure and remediation tracking	Medium

Confidence is high on the technical core of the issue because multiple strong sources align on the flaw mechanics, affected versions, and fixed releases.

Confidence is lower on exploitation prevalence because the reviewed source set did not show CISA KEV inclusion, confirmed in-the-wild exploitation, or a public weaponized proof of concept. The strongest supportable conclusion is that this is a genuine, remotely triggerable availability risk whose operational severity depends on whether a target validates attacker-controlled certificate chains at scale.

4. Vulnerability Details

According to the Go CNA and the Go issue tracker, the flaw occurs during chain building when certificate verification is presented with a large pool of candidate intermediates. The verification path does more work than intended while evaluating possible parent certificates, allowing an attacker to amplify the cost of a single verification attempt.

Because the affected surface is `Certificate.Verify` and chain-building logic beneath it, the vulnerability matters both to software that directly uses `crypto/x509` and to higher-level consumers of Go's TLS stack. In practical terms, an attacker does not need to achieve trust-chain success to impose cost; they only need to coerce a vulnerable application into processing a certificate set with many candidate intermediate certificates.

Reviewed public material supports an availability-impact conclusion only. There is no evidence in the primary sources that this flaw enables remote code execution, privilege escalation, authentication bypass, or confidentiality compromise on its own.

5. Attack Scenarios

1. A malicious remote TLS endpoint presents a certificate set containing a very large number of candidate intermediate certificates to a vulnerable Go client, proxy, or agent. Repeated handshakes force expensive chain-building work and can degrade service responsiveness.
2. An attacker targets internet-facing Go software that performs repeated outbound TLS validation to attacker-controlled infrastructure, such as scanners, collectors, API clients, or middleware. Even failed verification attempts may still consume disproportionate resources.
3. A custom Go application directly calls `x509.Certificate.Verify` with untrusted certificate material supplied by a remote user, upstream service, or imported bundle. Large intermediary pools can turn verification into a CPU exhaustion path.
4. Multi-tenant services that validate certificates on behalf of many customers or workflows may be especially susceptible if they do not rate-limit or isolate expensive verification operations.

6. Threat Landscape & Exploitation Status

- Public disclosure is broad across MSRC, the Go CNA CVE record, NVD, the Go advisory database, the Go security mailing list, and downstream distro trackers.
- The issue is scored as network-reachable with no required privileges or user interaction in NVD's current CVSS v3.1 record, which supports treating it as an externally triggerable denial-of-service concern.
- No reviewed source showed CISA KEV inclusion, a public exploitation campaign, or trusted reporting of in-the-wild abuse at the time of this review.
- The impact profile is still meaningful for certificate-heavy services because the flaw sits on a foundational verification path that can be hit before the application decides the certificate chain is invalid or untrusted.

7. Affected Products and Exposure

The most defensible exposure statement is that affected products are Go applications built with vulnerable standard-library releases and using certificate-chain verification paths that can process attacker-influenced intermediates.

Exposure Pattern	Why It Matters
Go TLS clients that connect to untrusted or attacker-reachable remote services	Remote peers can repeatedly force expensive chain verification during normal TLS negotiation.
Security products, crawlers, scanners, agents, proxies, gateways, and middleware	These systems often validate many external certificates and can amplify the operational impact of CPU-heavy verification.
Applications that directly invoke <code>x509.Certificate.Verify`</code> or populate <code>VerifyOptions.Intermediates`</code> from remote or semi-trusted inputs	Custom code may bypass safer assumptions and become the easiest place for a remote attacker to trigger the flaw.
Shared services that centralize certificate verification for many tenants or workflows	A single hot path can create broad blast radius if verification work is not isolated or rate limited.
Applications rebuilt on <code>Go 1.25.9`</code> , <code>Go 1.26.2`</code> , or later maintained releases	These are reduced-risk cases because the vulnerable logic has been fixed upstream.
Systems that rarely validate untrusted certificate chains or tightly constrain accepted intermediates	Lower exposure because attacker control over intermediary volume is limited.

Downstream exposure also includes distro-packaged Go runtimes and appliances that embed vulnerable Go releases. Debian's public tracker confirms package-level impact and remediation tracking across multiple release lines.

8. Mitigation and Workarounds

Priority actions:

- Upgrade Go toolchains and rebuild affected applications to `1.25.9``, `1.26.2``, or later maintained releases.
- Inventory Go-built TLS clients, agents, proxies, scanners, gateways, and certificate-processing services that may validate attacker-controlled certificate chains.
- Review application code paths that populate `VerifyOptions.Intermediates`` from remote, uploaded, or otherwise untrusted sources.
- Where immediate upgrades are not possible, add rate limiting, concurrency limits, timeout guards, or workload isolation around certificate-verification-heavy operations.
- Reduce unnecessary acceptance of large externally supplied intermediate bundles where business logic allows.
- Track distro or vendor package advisories if production dependencies come from operating-system repositories or vendor appliances instead of direct upstream Go releases.

9. Detection Engineering

Detection is primarily about vulnerable-version discovery and operational symptom visibility.

Signal	Telemetry Source	Analytic Goal
Go runtime versions older than <code>1.25.9`</code> or in the vulnerable <code>1.26.0`-1.26.1`</code> window	Asset inventory, SBOM, package management, container image scans	Identify systems that require patching or exposure review.
CPU / latency / worker saturation spikes during certificate validation	APM, service metrics, reverse-proxy telemetry, TLS handshake logs	Spot active or attempted resource-exhaustion behavior.
Repeated failed handshakes with oversized	TLS debug logs, packet capture, reverse-	Identify suspicious endpoints forcing

certificate chains from the same remote peers	proxy logs	expensive verification work.
Services that accept uploaded or remotely supplied certificate bundles	Application logs, API audit logs, custom telemetry	Prioritize custom verification paths for review and hunting.

9.1 Hunt Query - process and package discovery

```
// Identify Linux package-management or container telemetry that suggests vulnerable Go toolchains.
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "apt list --installed", "dpkg -l", "rpm -qa", "syft", "grype")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
| order by Timestamp desc
```

9.2 Hunt Query - handshake-pressure symptoms

```
// Pivot on systems with repeated TLS or x509 verification failures near latency / timeout spikes.
DeviceEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("SslHandshakeFailed", "TlsHandshakeFailed", "CertificateValidationFailed")
| summarize FailureCount=count() by DeviceName, bin(Timestamp, 15m)
| where FailureCount > 20
| order by FailureCount desc
```

10. Threat Hunting

Suggested hunt questions:

- Which Go applications in the environment were built with versions older than `1.25.9` or in the vulnerable `1.26.0`-`1.26.1` window?
- Which of those applications validate certificates from untrusted or semi-trusted remote parties?
- Are there recent CPU spikes, worker stalls, timeout bursts, or TLS handshake slowdowns tied to certificate validation?
- Do any logs or packet captures show remote peers repeatedly presenting unusually large certificate chains or intermediate bundles?
- Are internal scanners, proxies, or automation components repeatedly contacting the same suspicious TLS endpoints while showing elevated verification cost?
- Do any custom applications accept uploaded or remotely supplied certificate bundles and then call Go verification routines on them?

10.1 Hunt Query - suspicious peers driving repeated verification cost

```
// Hunt for repeated connections to the same remote endpoints preceding TLS failures or service timeouts.
DeviceNetworkEvents
| where Timestamp > ago(30d)
| where RemotePort in (443, 8443, 9443)
| summarize ConnectionCount=count() by DeviceName, RemoteIP, RemoteUrl, bin(Timestamp, 15m)
| where ConnectionCount > 50
| order by ConnectionCount desc
```

11. MITRE ATT&CK Mapping

No ATT&CK technique is assigned with high confidence for this intake review. The public evidence supports a resource-exhaustion / denial-of-service condition in certificate verification, but not a documented adversary tradecraft chain specific enough to justify a precise ATT&CK mapping without over-claiming.

12. Validation / Lab Testing

This intake review validated the report against published advisory, API, and upstream-release material rather than reproducing the issue locally. Analyst validation steps completed:

- Reviewed the MSRC Update Guide listing that triggered the intake.
- Queried the published CVE 5.2 record from the Go CNA for the authoritative description, version ranges, impacted routines, references, and CWE assignment.
- Queried the NVD JSON 2.0 API for severity, vector, and publication metadata.
- Reviewed the Go advisory `GO-2026-4947` for affected-package and fixed-version confirmation.
- Reviewed the upstream Go issue `#78282` for exploit-mechanics framing.
- Reviewed the Go security-release announcement and release-history text confirming fixes in `Go 1.25.9` and `Go 1.26.2`.
- Reviewed Debian tracker material for downstream packaging awareness.

No Lost Boys Cyber proof-of-concept execution was performed as part of this draft review.

13. Recommended Actions Summary

Priority	Owner	Action
Critical	Platform / application owners	Upgrade affected Go toolchains to `1.25.9`, `1.26.2`, or later and rebuild exposed applications.
High	Security engineering	Prioritize Go clients, proxies, agents, gateways, and middleware that validate attacker-reachable certificate chains.
High	Detection engineering	Add operational safeguards and telemetry around certificate-verification-heavy code paths where patching is delayed.
Medium	Threat hunting / SOC	Hunt for TLS verification latency spikes and repeated oversized certificate-chain presentations in relevant telemetry.
Medium	Asset management	Track downstream distro packaging status if production dependencies come from Debian or other vendor-managed images.

14. References

11. Microsoft Security Response Center, `CVE-2026-32280`
12. CVE 5.2 record for `CVE-2026-32280` (Go CNA)
13. NVD entry / JSON 2.0 API for `CVE-2026-32280`
14. Go advisory `GO-2026-4947`
15. Go issue tracker `#78282`
16. Go security release announcement for `1.26.2` / `1.25.9`
17. Go release history
18. Debian Security Tracker entry for `CVE-2026-32280`